

Digitale Codierungen | DigCod

Zusammenfassung

INHALTSVERZEICHNIS

1. Stellenwertsystem	3
1.1. Konversion	3
1.2. Nachkommastellen	3
1.3. Subtraktion durch Addition	3
2. Dualsystem	4
2.1. Arithmetik	4
2.1.1. Komplementbildung (Zweierkomplement)	4
2.1.2. Subtraktion	4
2.1.3. Addition	4
2.1.4. Multiplikation	4
2.1.5. Division	4
2.2. Wertebereich	5
2.3. Bit	5
2.4. Präfixe	5
2.4.1. Multiplikation/Division	5
3. Codierungen	6
3.1. Warum reicht eine einzige Zahlendarstellung nicht aus?	6
3.2. Vorzeichen	6
3.2.1. Betrag mit Vorzeichen	7
3.2.2. (b-1) Komplement / Einerkomplement	7
3.2.3. (b) Komplement / Zweierkomplement	7
3.2.4. Exzess-Codierung (Bias-Code)	8
3.3. Gleitkommazahlen	9
3.3.1. Fixkommazahl	9
3.3.2. Allgemeiner Wertebereich	10
3.3.3. Auflösung	10
3.3.4. Arithmetik	10
3.4. Gleitkomma	11
3.4.1. Sonderfälle	12
3.4.2. Präzision	12
3.4.3. Addition	12
3.4.4. IEEE 754 - Single vs. Double	13
3.5. Text	13
3.5.1. ASCII	14
3.5.2. Unicode	14
4. Boolesche Logik	17
4.1. Terme	17
4.2. Normalformen	17
4.2.1. Disjunktive Normalform	17
4.2.2. KV-Diagramm	18
4.3. NAND (Not AND)	18

4.4. Von Logik zur Arithmetik	18
4.5. Definition $\mathbb{Z}_2$	19
4.5.1. Bitfolge Darstellungen	19
4.6. Fun games	20
5. Wahrscheinlichkeit	20
5.1. Ergebnismenge	21
5.2. Eigenschaften	21
5.3. Wahrscheinlichkeitsdefinition nach Laplace	21
5.4. Kombinatorik	22
5.4.1. Permutation mit Wiederholung	22
5.4.2. Permutation ohne Wiederholung	23
5.4.3. Variation mit Wiederholung	23
5.4.4. Variation ohne Wiederholung	23
5.4.5. Kombination mit Wiederholung	23
5.4.6. Kombination ohne Wiederholung	24
5.4.7. Übersicht Auswahl	24
5.4.8. Hypergeometrische Verteilung	24
5.5. Bitfehler	24
5.6. Gesamtwahrscheinlichkeit	25
5.7. Binomialverteilung	25
5.8. Bedingte Wahrscheinlichkeit	26
5.8.1. Zusammenhang mit Multiplikationsregel	27
5.8.2. Bayes-Theorem	27
5.9. Unabhängigkeit von Ereignissen	27
6. Informationstheorie	28
6.1. Informationsgehalt	28
6.2. Entscheidungsgehalt	28
6.3. Entropie	29
6.4. Redundanz	29
6.5. Codierung der Zeichen	30
6.5.1. Codewortlänge	30
6.5.2. Mittlere Codewortlänge	30
6.5.3. Redundanz des Codes	30
6.5.4. Präfixcodes	30
6.6. Shannon'sches Codierungstheorem	31
6.7. Diskrete Quellen	31
6.7.1. Quellen ohne Gedächtnis	31
6.7.2. Quellen mit Gedächtnis	31
7. Quellencodierung	32
7.1. Huffman-Codierung	33
7.2. Run-Length-Encoding (RLE)	34
7.3. Lempel-Ziv-Codierung (LZ77)	34
7.4. Lempel-Ziv-Welch-Komprimierung (LZW)	35
7.4.1. Funktionsweise	36
7.5. Kompressionsrate	37
8. Qubit	37
9. CPU	37

1. STELLENWERTSYSTEM

Begriff	Bedeutung
Basis (Radix)	$E \in \mathbb{N}, R \geq 2$
Ziffernmenge	$T_R = \{0, 1, \dots, R-1\}$
Darstellung einer Zahl (Polynom)	$N_R = \sum_{i=0}^n d_i R^i$
Stellenwertigkeit an der Position i	$d_i \in Z_R, R^i$

Beispiele

Dezimalsystem – $R = 10$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ – $110_{10} = 0d110$	Oktalsystem – $R = 8$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7\}$ – $110_8 = 0o110 = 72_{10}$
Dualsystem – $R = 2$ – $Z_{10} = \{0, 1\}$ – $0110_2 = 0b0110 = 6_{10}$	Hexadezimalsystem – $R = 16$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$ – $2D_{16} = 0x2D = 45_{10}$

1.1. KONVERSION

Dezimal- zu Dualsystem:

- Rechts shift ist eine Division durch 2
- Es entsteht nur dann ein Rest, wenn die Bitstelle $2^0 = 1$ ist.

Beispiel: 25_{10}

Resultat: $0001\ 1001_2$

Zähler	Nenner	Resultat	Rest	
25	2	12	1	} ↑
12	2	6	0	
6	2	3	0	
3	2	1	1	
1	2	0	1	

Dual- zu Dezimalsystem bei Nachkommastellen:

- Links shift ist eine Multiplikation mit 2

Beispiel: 0.1875_{10}

Resultat: $0.001\ 1_2$

a	b	Resultat	Rest	
0.1875	2	0.375	0	} ↓
0.375	2	0.75	0	
0.75	2	1.5	1	
0.5	2	1	1	

1.2. NACHKOMMASTELLEN

Erweiterung der Darstellung einer Zahl: $N_R = d_n R^n + \dots + d_{10} R^0 + d_{-1} R^{-1} + \dots + d_{-m} R^{-m}$

Beispiel: $(101.01)_2 = 1 * 2^2 + 0 * 2^1 + 1 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2} = 4 + 1 + \frac{1}{4} = 5.25_{10}$

1.3. SUBTRAKTION DURCH ADDITION

Beispiel: $753 + 247 = 1000$

Bei 1000 einen Überlauf (mod 1000): $753 + 247 \equiv 0 \Leftrightarrow 753 \equiv -247 \pmod{1000}$

Dann wäre $620 - 247 \equiv 620 + 753 = 1373 \equiv 373 \pmod{1000}$

2. DUALSYSTEM

2.1. ARITHMETIK

2.1.1. Komplementbildung (Zweierkomplement)

- 1) Rechnen in n Bit $\Rightarrow \pmod{2^n}$
 - Übertrag aus dem MSB wird verworfen (gerechnet wird in $\mathbb{Z}_{2^n}$)
- 2) $-b$ als Zweierkomplement
 - Additives Inverses von $b \pmod{2^n}$ ist: $-b \equiv 2^n - b \pmod{2^n}$
 - Praktische Berechnung:
 - Bits invertieren: $\sim b$
 - $+1$ addieren
 - $2K(b) = \sim b + 1$

2.1.2. Subtraktion

Subtraktion wird durch Addition des Komplements ersetzt

$$a - b \equiv a + 2K(b) \pmod{2^n}$$

Beispiel 1 : $13_{10} - 5_{10}$

8 Bit, wobei $13_{10} = 0000\ 1101_2$, $5_{10} = 0000\ 0101_2$

Zweierkomplement von 5_{10} :

– invertieren: $1111\ 1010_2$

– $+1$: $1111\ 1011_2$

Addieren: $0000\ 1101_2 + 1111\ 1011_2 = 0001\ 0000\ 1000_2$

MSB-Übertrag weg: $0000\ 1000_2 = 8_{10}$

Also: $13_{10} - 5_{10} = 8_{10}$

2.1.3. Addition

TODO:

signed, unsigned

2.1.4. Multiplikation

TODO:

signed, unsigned

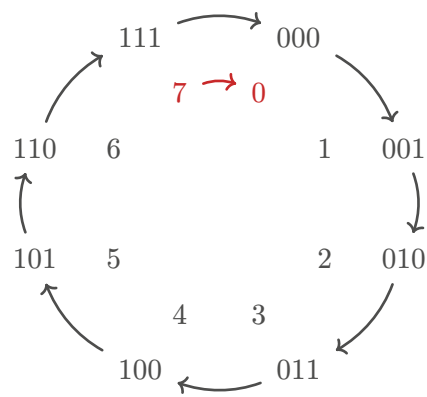
2.1.5. Division

TODO:

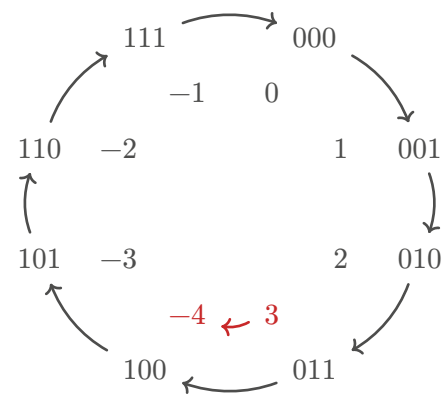
signed, unsigned

2.2. WERTEBEREICH

Graphische Veranschaulichung für Wortbreite von 3 Bit



unsigned: Überlauf von 7 auf 0



signed: Überlauf von 3 auf -4

2.3. BIT

Begriff	Bedeutung
set bit (gesetztes Bit)	1
cleared bit (gelöschtes Bit)	0
LSB	Least Significant Bit (Bit 0)
MSB	Most Significant Bit (Bit $n - 1$) in einer n -stelligen Binärzahl
Nibble	Binärzahl mit 4 Bit
Oktett	Binärzahl mit 8 Bit
Byte	Oktett

2.4. PRÄFIXE

Binär	Näherung	Binärpräfix	Dezimal	Dezimalpräfix
2^{10}	$1.024 \cdot 10^3$	Ki - Kibi	10^3	K - Kilo
2^{20}	$1.049 \cdot 10^6$	Mi - Mebi	10^6	M - Mega
2^{30}	$1.074 \cdot 10^9$	Gi - Gibi	10^9	G - Giga
2^{40}	$1.100 \cdot 10^{12}$	Ti - Tebi	10^{12}	T - Tera
2^{50}	$1.126 \cdot 10^{15}$	Pi - Pebi	10^{15}	P - Peta
2^{60}	$1.153 \cdot 10^{18}$	Ei - Exbi	10^{18}	E - Exa

2.4.1. Multiplikation/Division

- 1) Umformen in Potenzschreibweise
- 2) Addition der Exponenten
- 3) Umformen in Präfixschreibweise

Beispiel: $128K \cdot 64M = 2^7 \cdot 2^{10} \cdot 2^6 \cdot 2^{20} = 2^{17+26} = 2^{43} = 2^3 \cdot 2^{40} = 8T$

3. CODIERUNGEN

Erst wenn man die Codierung kennt, kann man Daten richtig interpretieren.

3.1. WARUM REICHT EINE EINZIGE ZAHLENDARSTELLUNG NICHT AUS?

- Negative Zahlen
 - Vorzeichen-Bit
 - Einerkomplement
 - Exzess-Code
- Sehr grosse und kleine Zahlen
 - Fixkomma ist unflexibel
 - 334 Milliarden + 0.00001
- Genauigkeit
 - 0.1 ist im Dualsystem nicht exakt darstellbar
 - Rundungsfehler unvermeidbar
- Nicht nur Zahlen
 - Textdarstellung
 - Zeichenkodierungen

3.2. VORZEICHEN

Jede Codierung optimiert eine andere Eigenschaft

- Betrag mit Vorzeichen: Intuition
- Einerkomplement: Einfache Negation
- Zweierkomplement: Arithmetische Effizienz
- Exzess: Vergleichbarkeit/Sortierung

<i>Binär</i>	<i>Betrag mit V.</i>	<i>EinerK.</i>	<i>ZweierK.</i>	$C_{ex,-8,4}$
0000	0	0	0	-8
0001	1	1	1	-7
0010	2	2	2	-6
0011	3	3	3	-5
0100	4	4	4	-4
0101	5	5	5	-3
0110	6	6	6	-2
0111	7	7	7	-1
1000	0	-7	-8	0
1001	-1	-6	-7	1
1010	-2	-5	-6	2
1011	-3	-4	-5	3
1100	-4	-3	-4	4
1101	-5	-2	-3	5
1110	-6	-1	-2	6

1111	-7	0	-1	7
------	----	---	----	---

3.2.1. Betrag mit Vorzeichen

Erstes Bit signalisiert, ob Zahl positiv (0) oder Negativ (1) ist.

Problem: Bekannte Rechenregeln funktionieren nicht.

$$\begin{aligned}
 -19_{10} + 1_{10} &= -18_{10} \\
 1001\ 0011_2 + 0000\ 0001_2 &= 1001\ 0100_2 = -20_{10}
 \end{aligned}$$

Beispiel 2 : Codierung Betrag mit Vorzeichen

Gegeben	Dezimal → Binär	Ergebnis
$x = -3_{10}$	$3_{10} = x011_2$	$-3_{10} = 1011_2$

Beispiel 3 : Decodierung Betrag mit Vorzeichen

Gegeben	Binär → Dezimal	Ergebnis
$x = 1001_2$	$x001_2 = 1_{10}$	$1001_2 = -1_{10}$

3.2.2. (b-1) Komplement / Einerkomplement

Von allen bits wird das Komplement gebildet.

Problem: $0000\ 0000_2 = 1111\ 1111_2 = 0_{10}$

Beispiel 4 : Codierung (b-1) Komplement

Gegeben	Dezimal → Binär	Komplement	Ergebnis
$x = -5_{10}$	$5_{10} = 0101_2$	$0101_2 \Rightarrow 1010_2$	$-5_{10} = 1010_2$

Beispiel 5 : Decodierung (b-1) Komplement

Gegeben	Komplement	Binär → Dezimal	Ergebnis
$x = 1011_2$	$1011_2 \Rightarrow 0100_2$	$0100_2 = 4_{10}$	$1011_2 = -4_{10}$

3.2.3. (b) Komplement / Zweierkomplement

Nach der Komplementbildung wird 1 addiert.

Beispiel 6 : Codierung (b) Komplement

Gegeben	Dezimal → Binär	Komplement	+1	Ergebnis
$x = -5_{10}$	$5_{10} = 0101_2$	$0101_2 \Rightarrow 1010_2$	$0101_2 \Rightarrow 1011_2$	$-5_{10} = 1011_2$

Beispiel 7 : Decodierung (b) Komplement

Gegeben	-1	Komplement	Binär → Dezi- mal	Ergebnis
$x = 1111_2$	$1111_2 \Rightarrow 1110_2$	$1110_2 \Rightarrow 0001_2$	$0001_2 = 1_{10}$	$1111_2 = -1_{10}$

3.2.4. Exzess-Codierung (Bias-Code)

Darstellung vorzeichenbehafteter Zahlen durch **Verschiebung des Wertebereichs**.

Statt negativer Zahlen wird ein **Offset (Bias)** addiert

Bedeutung 1 : $C_{ex,-B,n}(x)$

C_{ex} = Exzess-Codierung
 $-B$ = Bias (negativer Überhang zur 0)
 n = Länge der binären Schreibweise
 x = zu codierende Zahl

Gespeichert wird: $c = x + B$

Decodierung: $x = c - B$

Gegeben	Codierung	Decodierung
Basis $b = 2$ Wortbreite n Bias B typischerweise: $B = 2^{n-1} - 1$	$C_{ex,-B,n}(x) = x + B$ mit $x \in [-B, 2^n - 1 - B]$	$x = c - B$

Beispiel 8 : Codierung Bias-Code**Gegeben**

Wortbreite: $n = 8$ Bit
 Bias: $B = 2^7 - 1 = 127$
 Zu codierende Zahl: $x = -5$

Bias addieren

$$c = x + B = -5 + 127 = 122$$

Dezimal → Binär

$$122_{10} = 0111\ 1010_2$$

Ergebnis

$$C_{ex,-127,8}(-5) = 0111\ 1010_2$$

Beispiel 9 : Decodierung Bias-Code**Gegeben**

Wortbreite: $n = 8$ Bit
 Bias: $B = 2^7 - 1 = 127$
 Bitmuster: $C_{ex,-127,8} = 0111\ 1010_2$

Binär → Dezimal

$$0111\ 1010_2 = 122_{10}$$

Bias subtrahieren

$$x = c - B = 122 - 127 = -5$$

Ergebnis

$$0111\ 1010_2 \Rightarrow -5_{10}$$

3.3. GLEITKOMMAZAHLEN

3.3.1. Fixkommazahl

Skalierte Ganzzahl

<i>Pro</i>	<i>Contra</i>
Einfache Implementierung	Fester Wertebereich
Deterministische Genauigkeit	Feste Auflösung
Keine Exponentendarstellung	Überlauf bei grossen Zahlen
Schnelle Berechnung	Ungeeignet für stark unterschiedliche Grössenordnungen

Bedeutung 2 : $C_{FK,k,n}(x) = x \cdot 2^k$

C_{FK} = Fixkomma-Codierung

k = Anzahl Nachkommabits

n = Länge der binären Schreibweise - daraus ergibt sich die Anzahl der Vor-Kommastellen: $n - k$

x = zu codierende Zahl

I = Ganzzahl

Beispiel 10 : Codierung Fixkommazahl

Gegeben

Wortbreite: $n = 8$ Bit

Nachkommabits: $k = 4$

Zu codierende Zahl: $x = 3.25$

Skalieren

$$I = x \cdot 2^k = 3.25 \cdot 16 = 52$$

Dezimal $\rightarrow$ Binär

$$52_{10} = 0011\ 0100_2$$

Ergebnis

$$C_{FK,4,8}(3.25) = 0011\ 0100_2$$

Beispiel 11 : Decodierung Fixkommazahl

Gegeben

Wortbreite: $n = 8$ Bit

Nachkommabits: $k = 4$

Bitmuster: $C_{FK,4,8}(3.25)$

Binär $\rightarrow$ Dezimal

$$0011\ 0100_2 = 52_{10} = I$$

Skalieren

$$x = \frac{I}{2^k} = \frac{52}{16} = 3.25$$

Ergebnis

$$0011\ 0100_2 \Rightarrow 3.25_{10}$$

3.3.2. Allgemeiner Wertebereich

Wertebereich bei n Bit und k Nachkommabits

	Ganzzahlbereich	Reeller Bereich
unsigned	$I \in [0, 2^n - 1]$	$x \in \left[0, \frac{2^n - 1}{2^k}\right]$
signed	$I \in [-2^{n-1}, 2^{n-1} - 1]$	$x \in \left[-\frac{2^{n-1}}{2^k}, \frac{2^{n-1} - 1}{2^k}\right]$

Beispiel 12**Gegeben**

Wortbreite: $n = 8$ Bit

Nachkommabits: $k = 3$

Darstellung im Zweierkomplement (= inkl. Vorzeichen)

Ganzzahlbereich

$$I \in [-2^{8-1}, 2^{8-1} - 1] = [-128, 127]$$

Reeller Bereich

$$x = \frac{I}{2^k} = \frac{I}{8}$$

$$x_{\min} = \frac{-128}{8} = -16$$

$$x_{\max} = \frac{127}{8} = 15.875$$

Ergebnis

$$x \in [-16, 15.875]$$

3.3.3. Auflösung

Kleinst darstellbarer Schritt: $\Delta x = 2^{-k}$

Absoluter Fehler: $E_{\text{abs}} = |x_{\text{korrekt}} - x_{\text{gerundet}}|$

Relativer Fehler: $E_{\text{rel}} = \frac{|x_{\text{korrekt}} - x_{\text{gerundet}}|}{x_{\text{korrekt}}}$

3.3.4. Arithmetik

Addition, Subtraktion und Zweierkomplement sind wie bei Ganzzahlen

– Addiert man zwei Fixkommazahlen $z_0 + z_1$ mit $k_0 > k_1$, so muss z_1 um $k_0 - k_1$ Bits nach rechts geschoben werden.

Multiplikation und Division verschieben das Komma

– Multipliziert man zwei Fixkommazahlen $z_0 \cdot z_1 = z_2$, dann ist $k_2 = k_0 + k_1$

k muss für jede Zahl berücksichtigt werden

– Wird von den meisten Programmiersprachen kaum unterstützt

– Meist von Hand in Kommentaren

– Limitiert die Zahlen auf Bereiche, die zur Compilezeit bekannt sind

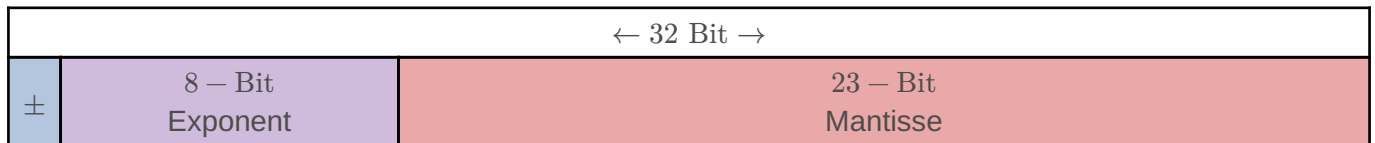
– unflexibel und fehleranfällig, aber performant

3.4. GLEITKOMMA

Bedeutung 3 : $\pm m \cdot 2^e$

m = Mantisse (Signifikand)

e = Exponent



$$\pm(1 + \text{Mantisse}) \cdot 2^{\text{Exponent} - 127}$$

Beispiel 13 : Codierung Gleitkommazahl

Gegeben

Zu codierende Zahl: $x = -42.625$

Vorzeichenbit bestimmen

x ist negativ, somit eine 1

Mantisse Dezimal → Binär

$\underbrace{101010}_{42} . \underbrace{101}_{0.625}$

Komma verschieben

$$101010.101_2 \cdot 2^0 = 1.01010101_2 \cdot 2^5$$

Runden

Bei Überfluss: falls vorherige Bit eine 1 ist, aufrunden, ansonsten kürzen (in diesem Beispiel nicht nötig).

Bias addieren

$$5 + 127 = 132$$

Exponent Dezimal → Binär

$$132_{10} \Rightarrow 1000\ 0100_2$$

Zusammensetzen

$$1 \mid 1000\ 0100 \mid (1)010\ 1010\ 1000\ 0000\ 0000\ 0000 \Rightarrow 1100\ 0010\ 0010\ 1010\ 1000\ 0000\ 0000\ 0000_2$$

Beispiel 14 : Decodierung Gleitkommazahl

Gegeben

Bitmuster: 0100 0001 0011 0110 0000 0000 0000 0000₂

Komponenten extrahieren

0 | 1000 0010 | (1)011 0110 0000 0000 0000 0000

Erstes Bit

0 = Zahl ist positiv

Exponent Binär → Dezimal

1000 0010₂ ⇒ 130₁₀

Bias subtrahieren

130 − 127 = 3

Mantisse Binär → Dezimal

011 0110 0000 0000 0000 0000₂ ⇒ 0.40625₁₀

Zusammensetzen

+1 · (1 + 0.40625) · 2³ = 11.25

3.4.1. Sonderfälle

Fall	Vorzeichenbit	Exponent	Mantisse	Beispielwert
Positive Null	0	0000 0000	000 0000 0000 0000 0000 0000	+0
Negative Null	1	0000 0000	000 0000 0000 0000 0000 0000	−0
Positive Unendlichkeit	0	1111 1111	000 0000 0000 0000 0000 0000	+∞
Negative Unendlichkeit	1	1111 1111	000 0000 0000 0000 0000 0000	−∞
NaN	0 oder 1	1111 1111	mindestens ein Bit 1	Nicht darstellbare Werte
Subnormale Zahlen	0 oder 1	0000 0000	mindestens ein Bit 1	≈ 1.4 · 10 ^{−45} (positiv)
Normalisierte Zahlen	0 oder 1	alles ausser 0000 0000 und 1111 1111	beliebig	Alle regulären Werte

3.4.2. Präzision

Viele Zahlen können nicht präzise dargestellt werden

- 0.1₁₀ = 0.0001100110011...
- Abbruch nach 23 Bits
- Beispiel: 0.1 + 0.2 ≠ 0.3
- Präzision für «Single Precision»: $\varepsilon \approx 2^{-23} \approx 10^{-7}$
- ε ist die kleinste relative Differenz nahe 1

3.4.3. Addition

- 1) Hidden Bit ergänzen
- 2) Exponenten vergleichen

- Wenn unterschiedlich: Bei kleinerer Zahl Mantisse nach rechts schieben
- 3) Vorzeichen berücksichtigen
 - Gleiche Vorzeichen: Addieren
 - Unterschiedliche Vorzeichen: Subtrahieren
- 4) Addition/Subtraktion der Signifikanden
- 5) Falls Carry = 1: Normalisieren
 - Erhöhe Exponent um 1
 - Schiebe Signifikand um 1 nach rechts
- 6) Hidden Bit entfernen

Beispiel 15 : Gleitkomma addition: $x = 1.5, y = 0.75$

1) Hidden Bit ergänzen

$$x' = 0 \mid 0111 \ 1111 \mid (1)100 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

$$y' = 0 \mid 0111 \ 1110 \mid (1)100 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

1) Exponent verschieben bei y'

$$x'' = 0 \mid 0111 \ 1111 \mid (1)100 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

$$y'' = 0 \mid 0111 \ 1111 \mid (0)110 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

1) Vorzeichen: beide positiv → Addition

2) Addition $z'' = x'' + y''$

$$z'' = 0 \mid 0111 \ 1111 \mid (10)010 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

1) Carry = 1: Normalisieren

$$z' = 0 \mid 1000 \ 0000 \mid (1)001 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

1) Hidden Bit entfernen

$$z = 0 \mid 1000 \ 0000 \mid 001 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

3.4.4. IEEE 754 - Single vs. Double

Format	Bits	Vorzeichen	Exponent	Mantisse	Bias
Single	32	1	8	23	-127
Double	64	1	11	52	-1023

- Grösserer Exponent → grösserer Wertebereich
- Grössere Mantisse → höhere Präzision
 - Single Precision: $\varepsilon \approx 2^{-23} \approx 10^{-7}$
 - Double Precision: $\varepsilon \approx 2^{-52} \approx 10^{-16}$
- Interpretation
 - Single ≈ 7 signifikante Dezimalstellen
 - Double $\approx 15^{-16}$ signifikante Dezimalstellen

3.5. TEXT

- Text besteht aus einer endlichen Folge von Zeichen: Buchstaben, Zahlen, Satzzeichen usw.

- Alle Zeichen stammen aus einer endlichen Menge Z , dem Zeichensatz (character set)
 - Jedem Zeichen kann eindeutig eine natürliche Zahl zugeordnet werden
- Ein Encoding (character encoding, Zeichenkodierung) ist eine bijektive Funktion E , die jedem Zeichen z eine natürliche Zahl zuordnet
 - $E : Z \rightarrow \{0, 1, \dots, |Z| - 1\}$
 - Text mit n Zeichen $z_0, \dots, z_{n-1}$ kann als endliche Folge von kodierten Zeichen beschrieben werden

3.5.1. ASCII

American Standard Code for Information Interchange

- Grösse des Zeichensatzes: $2^7 = 128 \rightarrow 7$ Bit (nicht 8 Bit!)
- 8-Bit-ASCII sind Extended ASCII-Varianten und nicht standardisiert, sondern Codepages – bspw.:
 - ISO-8859-1 (Latin-1)
 - für westeuropäische Sprachen mit zusätzlichen Buchstaben wie ä, ö, ü, é usw.
 - Windows-1252
 - Microsoft-Codepage – weitgehend wie ISO-8859-1 mit typografischen Zeichen wie €, “ usw.
 - Codepage 437
 - IBM-PC-Zeichensatz mit grafischen Symbolen, Linienzeichen und Sonderzeichen
- Enthält druckbare (darstellbare) Zeichen und (nicht darstellbare) Steuerzeichen (0x00=NUL, 0x07=BEL, ...)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL 00	SOH 01	STX 02	ETX 03	EOT 04	ENQ 05	ACK 06	BEL 07	BS 08	TAB 09	LF 0A	VT 0B	FF 0C	CR 0D	SO 0E	SI 0F
1	DLE 10	DC1 11	DC2 12	DC3 13	DC4 14	NAK 15	SYN 16	ETB 17	CAN 18	EM 19	SB 1A	ESC 1B	FS 1C	GS 1D	RS 1E	US 1F
2	SP 20	! 21	“ 22	# 23	\$ 24	% 25	& 26	‘ 27	(28	) 29	* 2A	+ 2B	, 2C	- 2D	. 2E	/ 2F
3	0 30	1 31	2 32	3 33	4 34	5 35	6 36	7 37	8 38	9 39	: 3A	; 3B	< 3C	= 3D	> 3E	? 3F
4	@ 40	A 41	B 42	C 43	D 44	E 45	F 46	G 47	H 48	I 49	J 4A	K 4B	L 4C	M 4D	N 4E	O 4F
5	P 50	Q 51	R 52	S 53	T 54	U 55	V 56	W 57	X 58	Y 59	Z 5A	[5B	\ 5C	] 5D	^ 5E	_ 5F
6	` 60	a 61	b 62	c 63	d 64	e 65	f 66	g 67	h 68	i 69	j 6A	k 6B	l 6C	m 6D	n 6E	o 6F
7	p 70	q 71	r 72	s 73	t 74	u 75	v 76	w 77	x 78	y 79	z 7A	{ 7B	 7C	} 7D	~ 7E	DEL 7F

3.5.2. Unicode

Globale Zeichennummerierung

- ASCII-kompatibel
- 1 bis 4 Byte pro Zeichen
- Häufige (westliche) Zeichen kurz
- Seltene (westliche) Zeichen länger
- Das erste Byte verrät, wie viele Bytes folgen

- Unicode = Codepoint
- UTF-8 = Bytecodierung

3.5.2.1. Struktur

<i>Codepoint-Bereich</i>	<i>Bytes</i>	<i>Bitmuster</i>
U+0000 - U+007F	1	0xxxxxxx
U+0080 - U+07FF	2	110xxxxx 10xxxxxx
U+0800 - U+FFFF	3	1110xxxx 10xxxxxx 10xxxxxx
U+10000 - U+10FFFF	4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

- Erstes Byte zeigt Länge
- Folgebytes beginnen mit 10
- ASCII-Zeichen (0-127) bleiben unverändert

3.5.2.2. Codierung

Gegeben: Unicode-Codepoint U

- 1) Bestimme den Wertebereich → Anzahl Bytes.
- 2) Schreibe U in Binärdarstellung.
- 3) Fülle die Bits in die x-Positionen des passenden Musters ein.
- 4) Wandle in Hex um.

Beispiel 16

Gegeben

Zeichen: ä

Unicode

$$U = E4_{16} = 228_{10} = 1110\ 0100_2$$

Benötigt 8 Bit → 2 Bytes

Muster einfügen

110xxxxx 10xxxxxx ⇒

11000011 10100100 ⇒ 1100 0011 1010 0100₂ = C3 A4₁₆

UTF-8(ä) = C3 A4₁₆

3.5.2.3. Decodierung

Gegeben: Bytefolge

- 1) Lies das erste Byte.
- 2) Bestimme anhand der führenden Bits die Länge.
- 3) Entferne die Strukturpräfixe (110, 10, etc.).
- 4) Füge die restlichen Bits zusammen.
- 5) Interpretiere als Codepoint.

Beispiel 17

Gegeben

1110 0010 1000 0010 1010 1100₂

Muster entfernen

11100010 10000010 10101100

1110xxxx 10xxxxxx 10xxxxxx ⇒

0010 000010 101100 ⇒ 0010 0000 1010 1100₂ = 20 AC₁₆ = €

3.5.2.4. Encodings**UTF-16**

- Meist 2 Bytes pro Zeichen
- Zeichen ausserhalb der BMP (Basic Multilingual Plane) benötigen in UTF-16 sogenannte Surrogate Pairs (4 Bytes)
- Nicht ASCII-kompatibel

Vorteil:

- Häufig effizient für asiatische Sprachen

Nachteil:

- Komplexere Handhabung

UTF-32

- Immer 4 Bytes pro Zeichen
- Direkter Zugriff auf Codepoints
- Sehr speicherintensiv

Encoding	Länge	Vorteil	Nachteil
UTF-8	1-4 Byte	ASCII-Kompatibel, effizient	Variable Länge
UTF-16	2-4 Byte	Teils effizient	Surrogate-Komplexität
UTF-32	4 Byte	Sehr einfach	Hoher Speicherbedarf

3.5.2.5. Basic Multilingual Plane (BMP)

Unicode ist als Zahlenraum definiert: $0 \leq U \leq 10\text{ FF FF}_{16}$

Das sind $1'114'111_{10}$ mögliche Codepoints. Dieser Raum ist in Planes (Ebenen) aufgeteilt.

Die BMP umfasst $U + 0000$ bis $U + \text{FFFF}$, also: $0 \leq U \leq 65535$. Das sind genau 16 Bit.

In der BMP liegen:

- ASCII
- Lateinische Schriftzeichen
- Griechisch
- Kyrrillisch
- Hebräisch
- Arabisch
- Viele asiatische Zeichen
- Mathematische Symbole
- Satzzeichen

4. BOOLSCHES LOGIK

- Eine Aussage ist entweder wahr (1) oder falsch (0)
- Aussagen können logisch verknüpft werden

Kommutativität

- $x \wedge y = y \wedge x$
- $x \vee y = y \vee x$

Distributivität

- $x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$

($x \wedge y$ schreibt man auch als xy oder $x \cdot y$ oder $x \cap y$,
 $x \vee y$ als $x + y$ oder $x \cup y$)

Assoziativität

- $(x \wedge y) \wedge z = x \wedge (y \wedge z)$
- $(x \vee y) \vee z = x \vee (y \vee z)$

Absorption

- $x \vee (x \wedge y) = x$
- $x \wedge (x \vee y) = x$

x	y	$x \wedge y$	$x \vee y$	$x \oplus y$	$\neg x$
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Dualitätsprinzip: Ersetze in einer wahren Gleichung $\wedge \leftrightarrow \vee$ und $0 \leftrightarrow 1$, Ergebnis bleibt wahr.

TODO:

[De-Morgan-Gesetze](#)

4.1. TERME

Begriff	Bedeutung
Literal	Variable oder Negation einer Variablen: x_3 (positiver Literal), $\neg x_3$ oder $\overline{x_3}$ (negatives Literal)
Konjunktionsterm	Konjunktion von Literalen: $\overline{x_1}x_3x_4 = \overline{x_1} \wedge x_3 \wedge x_4$
Disjunktionsterm	Disjunktion von Literalen: $\overline{x_1} \vee x_3 \vee x_4$
Minterm	Konjunktionsterm, der alle Parameter der Funktion enthält: $x_0\overline{x_1}\overline{x_2}x_3x_4$
Maxterm	Disjunktionsterm, der alle Parameter der Funktion enthält: $x_0 \vee \overline{x_1} \vee \overline{x_2} \vee x_3 \vee x_4$

4.2. NORMALFORMEN

Standardisierte Schreibweise. Vorteile:

- aus Wahrheitstabelle konstruierbar
- Vereinfachung systematisch möglich
- Umsetzung als Schaltung (AND-OR-Structure) direkt ableitbar

4.2.1. Disjunktive Normalform

in Term ist in DNF, wenn er eine ODER-Verknüpfung von UND-Verknüpfungen ist, z.B.:

TODO:

this is wrong

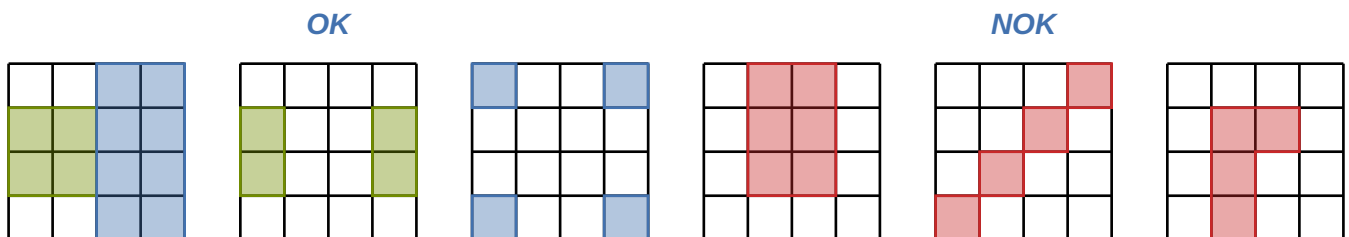
$$f(x, y, z) = (\neg x \wedge y) \vee (x \wedge z) = \neg x \wedge y \vee x \wedge z$$

Da $\wedge$ eine höhere Bindungsstärke als $\vee$ hat, sind die Klammern nicht zwingend notwendig. Die kanonische DNF (KDNF) ist die Disjunktion der Minterme.

4.2.2. KV-Diagramm

Je grösser die Blöcke, desto einfacher wird das Ergebnis. Dabei müssen aber bestimmte Regeln eingehalten werden:

- Die Blöcke müssen immer rechteckig sein und
- die Grösse einer Zweierpotenz haben, also 2, 4, 8, 16, 32, ...
- Die Blöcke können auch über den Rand hinaus gehen und mit der gegenüberliegenden Seite verbunden werden,
- Blöcke können sich teilweise überlappen. Das kann sinnvoll sein, wenn dadurch grössere Blöcke entstehen.
- Werte, die sowohl einfach als auch negiert vorkommen, werden gestrichen
- Ein Block wird nur berücksichtigt, wenn seine Einsen nicht vollständig in anderen Blöcken enthalten sind. Andernfalls entsteht ein nichtessentieller Term, der redundant ist, da andere Terme bereits die gleichen Variablenbelegungen abdecken und nicht weiter vereinfacht werden können.



4.3. NAND (NOT AND)

Universalbaustein

$$x \downarrow y = \overline{x \wedge y} = \overline{xy}$$

Sheffer stroke

- praktisch (in CMOS schnell sowie einfach aufzubauen)
- funktional vollständig: jede Boolesche Funktion lässt sich nur mit NAND realisieren.

x	y	$x \wedge y$	$x \downarrow y$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

OP	impl
NOT	$\overline{x} = \overline{x \wedge x} = x \downarrow x$
AND	$x \wedge y = \overline{x \downarrow y} = (x \downarrow y) \downarrow (x \downarrow y)$
OR	$x \vee y = \overline{\overline{x \vee y}} = \overline{\overline{x} \wedge \overline{y}} = \overline{(\overline{x} \downarrow \overline{x}) \wedge (\overline{y} \downarrow \overline{y})} = (x \downarrow x) \downarrow (y \downarrow y)$

4.4. VON LOGIK ZUR ARITHMETIK

Boolesche Logik realisiert binäre Addition

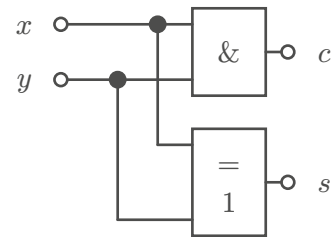
XOR bildet die Addition zweier Bits ab (im Zahlenraum mit nur 0 und 1, also mod 2), AND bildet den Übertrag ab

- $s = x \oplus y \rightarrow$ Addition mod 2
- $c = x \wedge y \rightarrow$ Übertrag

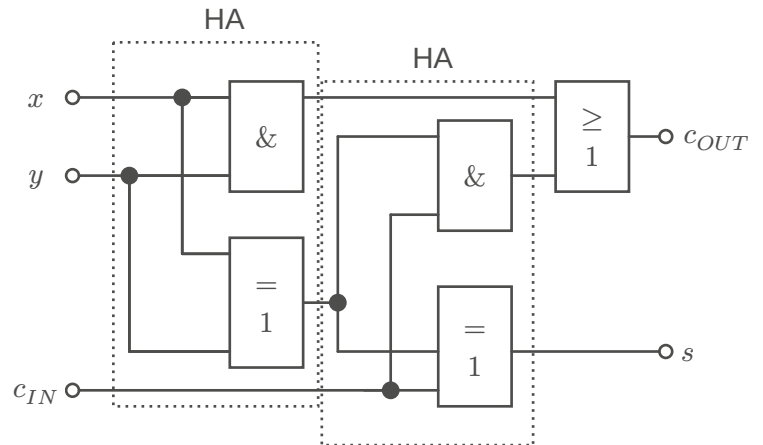
x	y	$x + y$	$x \wedge y$	$x \oplus y$
0	0	00	0	0
0	1	01	0	1
1	0	01	0	1
1	1	10	1	0

Halbaddierer (half adder)

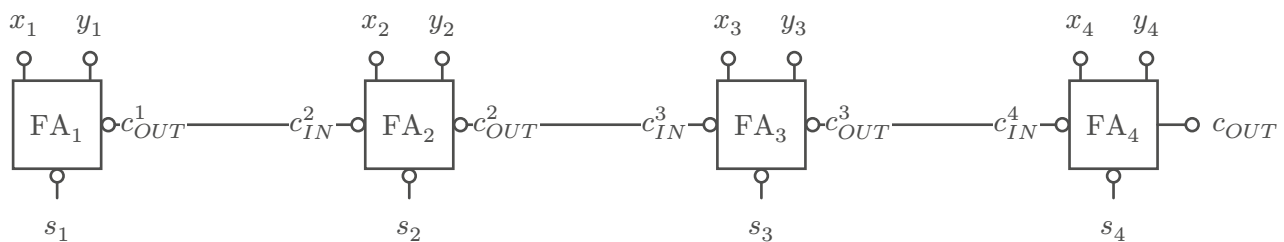
- Kann 2 einstellige Binärzahlen addieren
- Hat 2 Eingänge (x, y) und 2 Ausgänge (s, c)

**Volladdierer (full adder)**

- Hat 3 Eingänge (x, y, c_{IN})

**4-Bit Ripple carry adder**

- Jedes Bit muss auf das Carry-Bit des letzten Volladdierers warten

**Carry Look-Ahead adder**

- Reduziert propagations-delay durch komplexere hardware

4.5. DEFINITION $\mathbb{Z}_2$

$$\mathbb{Z}_2 = (\{0, 1\}, +, \cdot)$$

- Abgeschlossen: Jede Operation erzeugt wieder ein Element in $\mathbb{Z}_2$
- Es existiert das neutrale Element (0 für die Addition, 1 für die Multiplikation)
- Jedes Element ist sein eigenes additives Inverses.
- Addition und Multiplikation sind kommutativ und assoziativ.
- Es gilt das Distributivgesetz.

4.5.1. Bitfolge Darstellungen

Darstellung	Beispiel
Zahl	$0101_2 = 5_{10}$
Vektor	$(1, 0, 1)^T$
Polynom	$u^2 + 1$

4.5.1.1. Vektor

Beispiel 18 : Bitfolge als Vektor in $\mathbb{Z}_2^n$

$$v, w \in \mathbb{Z}_2^4, v = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix}, w = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \end{pmatrix}$$

$$v + w = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \end{pmatrix} \equiv \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix} \pmod{2}$$

4.5.1.2. Polynom

Polynome erlauben

- Verschiebungen elegant zu beschreiben
- Redundanz strukturiert zu erzeugen
- Generatorpolynome (zyklische Codes)
- CRC-Rechnung als Polynomdivision

Beispiel 19 : Bitfolge als Polynom in $\mathbb{Z}_2$

$$[1 \ 0 \ 0 \ 1 \ 0 \ 1]$$

$$= 1u^5 + 0u^4 + 0u^3 + 1u^2 + 0u^1 + 1u^0$$

$$= u^5 + u^2 + 1$$

u ist keine Zahl, u^k beschreibt «Bitposition k »

Polynomaddition in $\mathbb{Z}_2$

$$(u^3 + u + 1) + (u^3 + u^2)$$

$$= 2u^3 + u^2 + u + 1$$

$$= u^2 + u + 1$$

Polynommultiplikation in $\mathbb{Z}_2$

$$(u^2 + u + 1)(u + 1)$$

$$= u^3 + 2u^2 + 2u + 1$$

$$= u^3 + 1$$

4.6. FUN GAMES

[NAND Game](#)

[nand2tetris](#)

5. WAHRSCHEINLICHKEIT

In realen Systemen trifft Unsicherheit auf. Diese lässt sich nicht exakt vorhersagen, sondern nur statistisch beschreiben. Dafür verwenden wir **Wahrscheinlichkeit**.

Begriff	Bedeutung
Disjunkte Ereignisse	Ereignisse, die sich gegenseitig ausschliessen. Beispiel: z ist gerade oder ungerade
Zufallsvorgang	Ein Vorgang mit mehreren möglichen Ergebnissen, dessen Ausgang nicht sicher vorhergesagt werden kann
Zufallsexperiment	Zufallsvorgang, der geplant ist und kontrolliert abläuft

5.1. ERGEBNISMENGE

Begriff	Bedeutung
Ergebnismenge	Die Ergebnismenge eines Zufallsvorgangs umfasst alle möglichen Ausgänge des Experiments. Sie wird mit dem Symbol Ω (Omega) bezeichnet. Die Anzahl aller möglichen Ergebnisse der Ergebnismenge wird mit $ \Omega $ bezeichnet.
Ergebnis	Ein einzelner möglicher Ausgang $\omega \in \Omega$ heisst Ergebnis
Ereignis	Ein Ereignis ist eine Teilmenge der Ergebnismenge $A \subset \Omega$

Beispiel 20

Würfel: $\Omega = \{1, 2, 3, 4, 5, 6\}$

Werfen einer Münze so lange, bis Kopf erscheint: $\Omega = \{K, ZK, ZZK, ZZZK, \dots\}$

5.2. EIGENSCHAFTEN

Für jedes Ereignis A gilt $0 \leq P(A) \leq 1$.

Begriff	Eigenschaft
Sicheres Ereignis	Die Wahrscheinlichkeit der gesamten Ergebnismenge ist $P(\Omega) = 1$
Unmögliches Ereignis	Die Wahrscheinlichkeit der leeren Menge ist $P(\emptyset) = 0$
Additionsregel	Für zwei Ereignisse A und B : $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ Für disjunkte Ereignisse: $P(A \cup B) = P(A) + P(B)$
Gegenereignis	Wird notiert als $\bar{A}$ und hat die Eigenschaft $P(\bar{A}) = 1 - P(A)$.

Wahrscheinlichkeit, dass A_1 oder A_2 auftritt: $P(A_1 \vee A_2) = P(A_1) + P(A_2)$

Wahrscheinlichkeit, dass zuerst A_1 und dann A_2 auftritt: $P(A_1 \wedge A_2) = P(A_1) \cdot P(A_2)$

5.3. WAHRSCHEINLICHKEITSDEFINITION NACH LAPLACE

Wenn ein Experiment eine Anzahl verschiedener und gleich möglicher Ausgänge hervorbringen kann und einige davon als günstig anzusehen sind, dann ist die Wahrscheinlichkeit eines günstigen Ausganges gleich dem Verhältnis der Anzahl der günstigen zur Anzahl der möglichen Ausgänge.

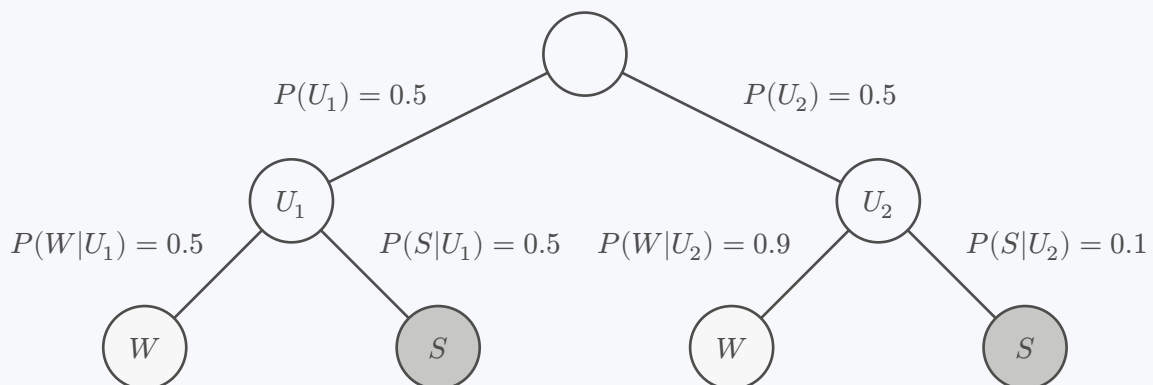
$$P(E) = \frac{\text{Anzahl günstiger Ergebnisse}}{\text{Anzahl aller möglichen Ergebnisse}} = \frac{|E|}{|\Omega|}$$

Dabei gilt:

- Ω : Ergebnismenge (alle möglichen Ergebnisse)
- $E \subseteq \Omega$: betrachtetes Ereignis
- $|E|$: Anzahl günstiger Ergebnisse
- $|\Omega|$: Anzahl aller möglichen Ergebnisse

Beispiel 21 : Urnen

Es gibt zwei Urnen, U_1 und U_2 . Urne U_1 enthält 5 schwarze und 5 weiße Kugeln und Urne U_2 enthält 9 schwarze und 1 weiße Kugel. Die Wahrscheinlichkeit eine Kugel aus U_1 oder U_2 zu ziehen, ist gleich gross (50/50). Wie gross ist nun die Wahrscheinlichkeit eine weiße Kugel zu ziehen?



$$\begin{aligned}
 P(W|U_1) \wedge P(U_1) \vee P(W|U_2) \wedge P(U_2) &= P(W|U_1) \cdot P(U_1) + P(W|U_2) \cdot P(U_2) \\
 &= 0.5 \cdot 0.5 + 0.5 \cdot 0.1 \\
 &= 0.25 + 0.05 \\
 &= 0.3
 \end{aligned}$$

Die Wahrscheinlichkeit beträgt also 30%, eine weiße Kugel zu ziehen unter der Annahme, dass jede Urne mit gleicher Wahrscheinlichkeit gewählt wird.

5.4. KOMBINATORIK

Laplace-Wahrscheinlichkeit erfordert Berechnung von Anzahlen. **Kombinatorik** ist die Mathematische Technik hierfür.

	Wiederholung	Keine Wiederholung
Anordnung	Permutation mit Wiederholung	Permutation ohne Wiederholung
Geordnete Auswahl	Variation mit Wiederholung	Variation ohne Wiederholung
Ungeordnete Auswahl	Kombination mit Wiederholung	Kombination ohne Wiederholung

5.4.1. Permutation mit Wiederholung

Anordnung von n Objekten, die **nicht** alle voneinander unterscheidbar sind (s Subsets k mit gleichen Objekten).

$$\frac{n!}{k_1!k_2!\dots k_s!}$$

Beispiel 22 : Anordnung drei roter und zwei blauer Kugeln

$$s = 2, n = 5, k_1 = 3, k_2 = 2, \text{Anzahl} = \frac{(5 \cdot 4 \cdot 3 \cdot 2 \cdot 1)}{(3 \cdot 2 \cdot 1)(2 \cdot 1)} = 10$$

5.4.2. Permutation ohne Wiederholung

Anordnung von n Objekten, die alle unterscheidbar sind.

$$n!$$

Beispiel 23: Anordnung fünf verschiedenfarbiger Kugeln

$$\text{Anzahl} = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 5!$$

5.4.3. Variation mit Wiederholung

- Reihenfolge spielt eine Rolle
- Elemente dürfen mehrfach vorkommen

$$n^k$$

Beispiel 24: PIN-Code mit 4 Ziffern

0000

1234

9876

Jede Position hat 10 Möglichkeiten

$$\text{Anzahl} = 10 \cdot 10 \cdot 10 \cdot 10 = 10^4$$

5.4.4. Variation ohne Wiederholung

- Reihenfolge spielt eine Rolle
- Elemente dürfen **nicht** mehrfach vorkommen

$$\frac{n!}{(n-k)!} = \prod_n^{n-k+1} n$$

Beispiel 25: 1., 2. und 3. Platz aus 10 Teilnehmern

$$\text{Anzahl} = 10 \cdot 9 \cdot 8$$

5.4.5. Kombination mit Wiederholung

- Reihenfolge spielt **keine** Rolle
- Elemente dürfen mehrfach vorkommen

$$\binom{n+k-1}{k} = \frac{(n+k-1)!}{(n-1)! \cdot k!}$$

Beispiel 26: Es sollen drei von fünf verschiedenfarbigen Kugeln gezogen und wieder zurückgelegt werden.

$$\text{Anzahl} = \binom{5+3-1}{3} = \binom{7}{3} = 35$$

5.4.6. Kombination ohne Wiederholung

- Reihenfolge spielt **keine** Rolle
- Elemente dürfen **nicht** mehrfach vorkommen

$$\binom{n}{k} = \frac{n!}{(n-k)!k!} = \frac{n!}{k!(n-k)!} = \frac{\prod_n^{n-k+1} n}{k!}$$

Beispiel 27 : Lotto 6 aus 42

$$\text{Anzahl} = \binom{42}{6}$$

5.4.7. Übersicht Auswahl

Anzahl A der Möglichkeiten			
n Optionen k Auswählen		Beachtung der Reihenfolge	
		✓	×
Wiederholung erlaubt	✓	$A = n^k$	$A = \binom{n+k-1}{k}$
	×	$A = n(n-1)\dots(n-k+1) = \frac{n!}{(n-k)!}$	$A = \frac{n!}{k!(n-k)!} = \binom{n}{k}$

5.4.8. Hypergeometrische Verteilung

Die hypergeometrische Verteilung ist ein Modell, das verwendet wird, um die Wahrscheinlichkeit $P(k)$ von k Erfolgen (gewünschten Ergebnissen) aus den K gesamt möglichen Erfolgen aus N Objekten in n Ziehungen zu berechnen.

$$P(k) = \frac{\binom{K}{k} \cdot \binom{N-K}{n-k}}{\binom{N}{n}}$$

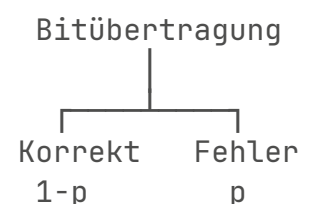
Beispiel 28

TODO:

5.5. BITFEHLER

Ein Bitfehler bezeichnet die inkorrekte Wiedergabe eines Bits während der Datenübertragung oder -speicherung. Das bedeutet, dass ein Bit von 0 zu 1 oder von 1 zu 0 fälschlicherweise geändert wird.

Die Wahrscheinlichkeit, dass ein Datenblock der Grösse n Bits bei einer Fehlerrate von p fehlerhaft ist, kann mit folgender Formel berechnet werden:



$$P(\text{fehlerhaft}) = 1 - (1 - p)^n$$

Wahrscheinlichkeit für genau k Fehler in einem Datenblock mit n Bits bei Bitfehlerrate p (Binomialverteilung):

$$P(k) = \binom{n}{k} \cdot p^k \cdot (1 - p)^{n-k}$$

Beispiel 29 : Bitfehler

Gegeben sei eine Bitfehlerrate von 10^{-5} . Wie gross ist die Wahrscheinlichkeit, dass ein Datenblock mit einer Grösse von 150 kbit fehlerhaft ist?

$$P(\text{fehlerhaft}) = 1 - (1 - 10^{-5})^{150 \cdot 10^3} \approx 0.77687$$

Was ist die Wahrscheinlichkeit für genau 2 Fehler?

$$P(2) = \binom{150 \cdot 10^3}{2} \cdot (10^{-5})^2 \cdot (1 - 10^{-5})^{150 \cdot 10^3 - 2} \approx 0.25102$$

5.6. GESAMTWAHRSCHEINLICHKEIT

Jede mögliche Fehleranordnung hat die Wahrscheinlichkeit

$$p^k (1 - p)^{n-k}$$

Es gibt $\binom{n}{k}$ solche Anforderungen. Darum gilt

$$P(X = k) = \overbrace{\binom{n}{k}}^{\text{Anzahl Kombinationen}} \underbrace{p^k}_{k \times \text{Erfolg}} \overbrace{(1 - p)^{n-k}}^{n-k \times \text{Misserfolg}}$$

Beispiel 30 : Schraubenproduktion

Bei einer Schraubenproduktion ist jede 10e Schraube fehlerhaft. Was ist die Wahrscheinlichkeit, dass 2 von 3 Schrauben OK sind?

$$P(X = 2) = \binom{3}{2} \cdot \left(\frac{9}{10}\right)^2 \cdot \left(1 - \frac{9}{10}\right)^{3-2} = \binom{3}{2} \cdot \frac{81}{100} \cdot \frac{1}{10} = 3 \cdot \frac{81}{1000} = 0.243$$

TODO:

example (slides 27)

TODO:

Restfehlerwahrscheinlichkeit

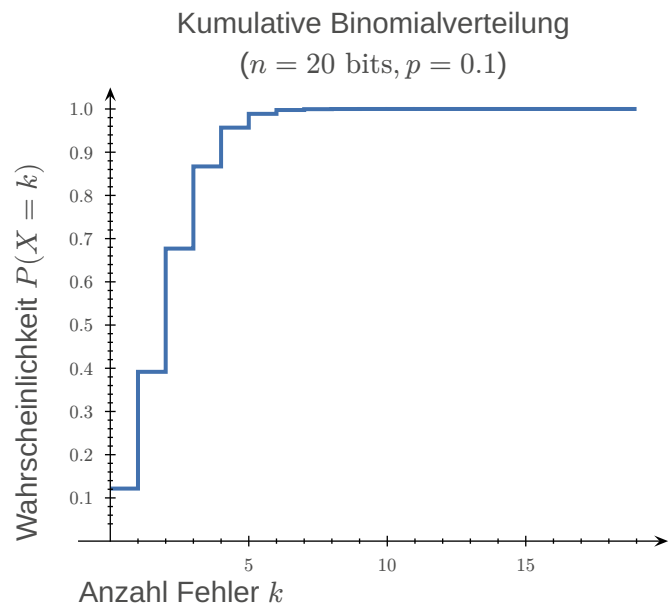
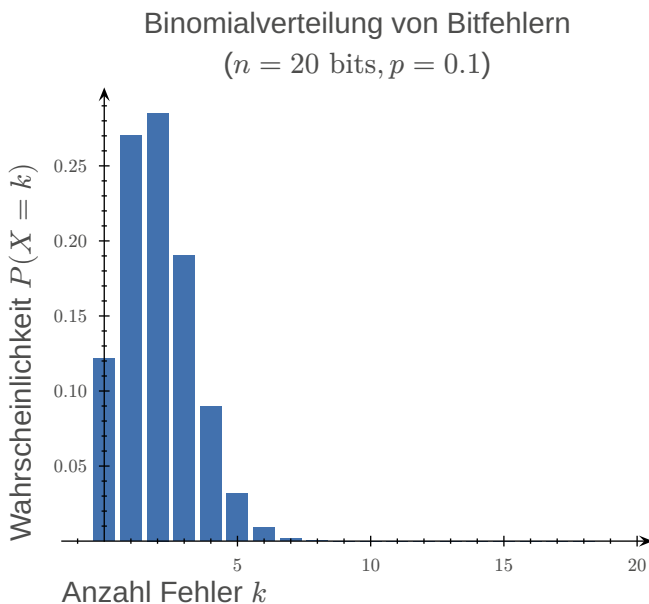
5.7. BINOMIALVERTEILUNG

$$E(X) = np$$

Die Binomialverteilung beschreibt, wie wahrscheinlich es ist, dass bei n unabhängigen Versuchen genau k Erfolge auftreten.

In unserem Kontext bedeutet das:

- n = Anzahl übertragener Bits
- p = Bitfehlerwahrscheinlichkeit
- k = Anzahl Fehler



5.8. BEDINGTE WAHRSCHEINLICHKEIT

Wie wahrscheinlich ist Ereignis A , wenn Ereignis B bereits eingetreten ist? Formelle Schreibweise: $P(A|B)$.

Allgemein gilt für die bedingte Wahrscheinlichkeit:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Voraussetzung ist dabei, dass gilt:

$$P(B) > 0$$

Die Formel bedeutet:

- Im Nenner steht die Wahrscheinlichkeit der Bedingung B .
- Im Zähler steht die Wahrscheinlichkeit, dass sowohl A als auch B eintreten.

Man betrachtet also nur noch den Teil des Wahrscheinlichkeitsraums, in dem B gilt, und fragt dann, wie gross darin der Anteil der Fälle ist, in denen zusätzlich A eintritt.

Daraus folgt die **Multiplikationsregel**:

$$P(A \cap B) = P(A|B) \cdot P(B)$$

Beispiel 31 : Würfelwurf

A = «Die gewürfelte Zahl ist gerade»

B = «Die gewürfelte Zahl ist grösser als 3»

$$A = \{2, 4, 6\}, B = \{4, 5, 6\}, P(A) = P(B) = \frac{1}{2}$$

$$A \cap B = \{4, 6\}, P(A | B) = \frac{2}{3}$$

Die Wahrscheinlichkeit für «gerade Zahl» ist also unter der Bedingung «größer als 3» nicht mehr $\frac{1}{2}$, sondern $\frac{2}{3}$.

5.8.1. Zusammenhang mit Multiplikationsregel

Aus der Definition folgt direkt eine wichtige Umformung:

$$P(A \cap B) = P(A|B) \cdot P(B)$$

Ebenso gilt auch:

$$P(A \cap B) = P(B|A) \cdot P(A)$$

5.8.2. Bayes-Theorem

Setzt man die beiden Ausdrücke gleich, erhält man

$$P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$$

Durch Umstellen ergibt sich das **Bayes-Theorem**:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Diese Formel erlaubt es, Wahrscheinlichkeiten umzukehren: Aus der Wahrscheinlichkeit von B unter der Bedingung A kann die Wahrscheinlichkeit von A unter der Bedingung B berechnet werden.

5.8.2.1. Satz der totalen Wahrscheinlichkeit

Angenommen ein Ereignis B kann durch mehrere verschiedene Ursachen entstehen. Seien

$$A_1, A_2, \dots, A_n$$

Ereignisse, die

- sich gegenseitig ausschließen
- zusammen den gesamten Ereignisraum bilden

Dann gilt für jedes Ereignis B :

$$P(B) = P(B|A_1) \cdot P(A_1) + P(B|A_2) \cdot P(A_2) + \dots + P(B|A_n) \cdot P(A_n)$$

Diese Formel nennt man den **Satz der totalen Wahrscheinlichkeit**. Sie beschreibt die Gesamtwahrscheinlichkeit eines Ereignisses als Summe aller möglichen Fälle, in denen es entstehen kann.

5.9. UNABHÄNGIGKEIT VON EREIGNISSEN

Zwei Ereignisse A und B heissen unabhängig, wenn das Eintreten des einen Ereignisses keinen Einfluss auf die Wahrscheinlichkeit des anderen hat. Dann gilt:

$$P(A|B) = P(A)$$

Das bedeutet: Auch wenn B bereits eingetreten ist, bleibt die Wahrscheinlichkeit von A unverändert. Setzt man das in die Multiplikationsregel ein, erhält man:

$$P(A \cap B) = P(A) \cdot P(B)$$

6. INFORMATIONSTHEORIE

Die Informationstheorie versucht, mathematisch zu messen:

- wie viel Information Ereignisse enthalten
 - ein sehr erwartbares Ereignis liefert wenig Information
 - ein überraschendes Ereignis liefert viel Information
- wie unsicher eine Informationsquelle ist
- wie effizient Informationen codiert werden können.

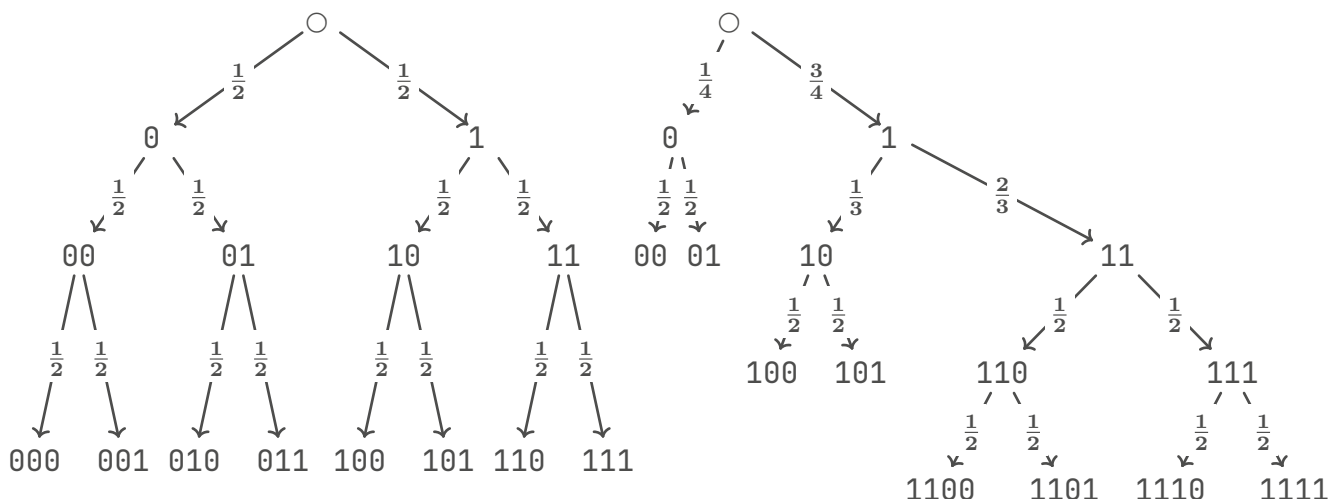
Die **Entropie** ist dabei die zentrale Grösse, die die durchschnittliche Informationsmenge einer Quelle beschreibt.

Die Einheit der Information ist das **Bit**.

6.1. INFORMATIONSGEHALT

Der Informationsgehalt $I(x_k)$ eines Symbols x_k ist ein Mass dafür, wie viel Information das Symbol trägt, basierend auf seiner Wahrscheinlichkeit des Auftretens $p(x_k)$.

$$I(x_k) = \log_2 \left(\frac{1}{p(x_k)} \right) = -\log_2(p(x_k))$$



6.2. ENTSCHEIDUNGSGEHALT

Der Entscheidungsgehalt H_0 einer Quelle gibt an, wie viel Information im Durchschnitt benötigt wird, um ein Ereignis aus N gleich wahrscheinlichen unterschiedlichen Ereignissen zu identifizieren.

$$H_0 = \log_2(N)$$

TODO:

Gleichwahrscheinliche Ereignisse

- Angenommen eine Quelle besitzt N mögliche und gleichwahrscheinliche Symbole.

Wahrscheinlichkeit eines Symbols

- Bei gleichwahrscheinlichen Symbolen gilt: $p(x) = \frac{1}{N}$

Informationsgehalt eines Ereignisses: $I(x) = -\log_2 P(x)$

- Einsetzen von $p(x) = \frac{1}{N}$: $I(x) = -\log_2 \left(\frac{1}{N} \right) = \log_2 N$

Ergebnis: $I(x) = \log_2 N = H_0$

– für gleichwahrscheinliche Ereignisse.

⇒ Der Entscheidungsgehalt H_0 ist ein Spezialfall des Informationsgehalts $I(x)$ für gleichwahrscheinliche Ereignisse.

6.3. ENTROPIE

Die Entropie $H(X)$ beschreibt den durchschnittlichen Informationsgehalt / durchschnittliche Unsicherheit einer Quelle.

$$H(X) = \sum_{k=1}^N p(x_k) \cdot I(x_k) = \sum_{k=1}^N p(x_k) \cdot \log_2 \left(\frac{1}{p(x_k)} \right) = - \sum_{k=1}^N p(x_k) \cdot \log_2(p(x_k))$$

$p(x) = 1$	$p(x) = 0$	$0 \leq H(X) \leq \log_2 N$
– Ergebnis ist sicher – $I(x) = 0$	– Ergebnis tritt nie auf – Trägt 0 zur Entropie bei	– Minimum: deterministische Quelle – Maximum: gleichverteilte Quelle

Beispiel 32 : $H(X) = -(0.99 \cdot \log_2 0.99 + 0.01 \cdot \log_2 0.01) = 0.014 + 0.066 \approx 0.080$ Bit

- A liefert fast keine Information, weil es praktisch immer kommt.
- B liefert sehr viel Information, weil es extrem selten ist.
 - Aber: B kommt nur 1% der Zeit, deshalb ist der durchschnittliche Informationsgehalt klein.

⇒ Ergebnis: ca. 0.08 Bit pro Symbol

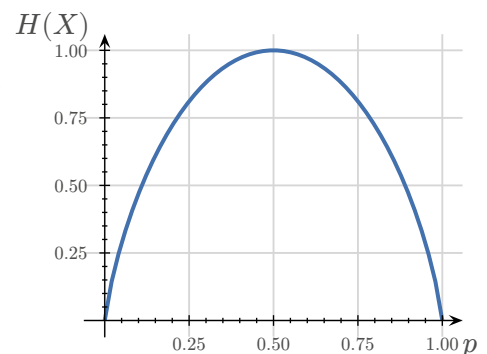
Zeichen	Wahrscheinlichkeit
A	0.99
B	0.01

6.4. REDUNDANZ

Die Redundanz R_q beschreibt den Anteil vorhersehbarer Information / den Unterschied zwischen dem maximal möglichen Entscheidungsgehalt und der tatsächlichen Entropie der Quelle.

$$R_q = R_{\text{abs}} = H_0 - H(X) \quad [\text{Bit}/\text{Zeichen}]$$

$$R_{\text{rel}} = \frac{R_{\text{abs}}}{H_0} = \frac{H_0 - H(X)}{H_0} = 1 - \frac{H(X)}{H_0} \quad [\%]$$



Hohe Redundanz

Quelle stark vorhersehbar

Geringe Redundanz

Quelle nahe maximaler Unsicherheit

Eine Quelle mit hoher Redundanz enthält viele regelmässige Strukturen oder Abhängigkeiten. Diese können genutzt werden, um Daten effizienter zu codieren oder zu komprimieren.

6.5. CODIERUNG DER ZEICHEN

6.5.1. Codewortlänge

Idealerweise sollte die Länge eines Codeworts dem Informationsgehalt des Symbols entsprechen:

$$L(x_k) \approx I(x_k) \approx -\log_2 p(x_k)$$

Da Codewörter jedoch aus einer **ganzen Anzahl Bits** bestehen müssen, wird aufgerundet:

$$L(x_k) = \lceil I(x_k) \rceil = \lceil -\log_2 p(x_k) \rceil$$

6.5.2. Mittlere Codewortlänge

Die mittlere Codewortlänge $L(X)$ ist definiert als der gewichtete Durchschnitt der Längen der Codewörter, wobei jedes Gewicht der Auftretenswahrscheinlichkeit des entsprechenden Symbols gleich ist.

$$L(X) = \sum_{k=1}^N p(x_k) \cdot L(x_k)$$

Es gilt für jeden Code $H(X) \leq L(X)$

Beispiel 33

Folgende Zeichen mit entsprechenden Codewörter, deren Länge und ihren Wahrscheinlichkeiten sind gegeben:

Zeichen	Codewort	Wahrscheinlichkeit p	Länge
a	0	0.3	1
b	110	0.1	3
c	1111	0.1	4
d	1110	0.2	4
e	10	0.3	2

Nun berechnen wir die mittlere Codewortlänge L mit der obigen Formel:

$$\begin{aligned}
 L &= (0.3 \cdot 1) + (0.1 \cdot 3) + (0.1 \cdot 4) + (0.2 \cdot 4) + (0.3 \cdot 2) \\
 &= 0.3 + 0.3 + 0.4 + 0.8 + 0.6 \\
 &= 2.4 \text{ bit}
 \end{aligned}$$

6.5.3. Redundanz des Codes

Die Redundanz des Codes R_c ist die Differenz zwischen der mittleren Codewortlänge und der Entropie der Quelle.

$$R_c = L - H(x)$$

Interpretation: Beschreibt, wie ineffizient die Codierung ist / Verlust durch nicht-optimalen Code.

6.5.4. Präfixcodes

Ein Code besitzt die **Präfixeigenschaft**, wenn kein Codewort der Anfang eines anderen Codeworts ist.

Symbol	Code
A	0

Präfixcodes sind wichtig, weil sie eine **eindeutige und sofortige Decodierung** ermöglichen.

<i>Symbol</i>	<i>Code</i>
B	10
C	110
D	111

6.6. SHANNON'SCHES CODIERUNGSTHEOREM

Das Shannon-Theorem beschreibt die theoretischen Grenzen der Datenkompression. Für jede Informationsquelle mit mittlerer Codewortlänge $L(X)$ und deren Entropie $H(X)$ gilt:

$$H(X) \leq L(X) < H(X) + 1$$

- Entropie ist die theoretische Mindestlänge eines Codes / Grenze der Kompression
- Praktische Codes können dieser Grenze sehr nahe kommen.

6.7. DISKRETE QUELLEN

6.7.1. Quellen ohne Gedächtnis

Diskrete Quellen ohne Gedächtnis haben Symbole, die unabhängig von vorherigen Symbolen auftreten. Jedes Symbol x_k aus einem endlichen Set tritt mit einer Wahrscheinlichkeit $p(x_k)$ auf. Verbundwahrscheinlichkeit für die beiden Zeichen x_i und x_{i+1} lautet:

$$p(x_i, x_{i+1}) = p(x_i) \cdot p(x_{i+1})$$

6.7.2. Quellen mit Gedächtnis

In der Praxis sind nur wenige Datenquellen vollständig gedächtnislos. Häufig hängt die Wahrscheinlichkeit eines Zeichens vom vorhergehenden Zeichen ab. Solche Kontextabhängigkeiten lassen sich mit bedingten Wahrscheinlichkeiten beschreiben: $p(x_{i+1} | x_i)$

Beispiel 34 : Zeichenfolgen

In deutschen und englischen Texten folgt auf den Buchstaben «q» praktisch immer «u».

$$p(u|q) \approx 1$$

6.7.2.1. Entropie

Für zwei aufeinanderfolgende Zeichen gilt:

$$H(X, Y) = \sum_{k=1}^N \sum_{i=1}^N p(x_k, y_i) \cdot \log_2 \left(\frac{1}{p(x_k, y_i)} \right)$$

Mit

$$p(x_k, y_i) = p(x_k) \cdot p(y_i | x_k)$$

ergibt sich

$$H(X, Y) = H(X) + H(Y | X)$$

Interpretation

- $H(X)$: Unsicherheit über das erste Zeichen

- $H(X, Y)$: «Verbundentropie» = Unsicherheit über zwei aufeinanderfolgende Zeichen
- $H(Y | X)$: verbleibende Unsicherheit über Y wenn X bereits bekannt ist

Da Kontextinformation Unsicherheit reduziert, gilt:

$$H(Y | X) \leq H(Y)$$

6.7.2.2. Redundanz

$$R_{\text{abs,oG}} = H_0 - H(Y)$$

$$R_{\text{abs,mG}} = H_0 - H(Y | X)$$

$$R_{\text{abs,oG}} \leq R_{\text{abs,mG}}$$

Interpretation: beschreibt, wie viel «überflüssige Struktur» in der Quelle steckt / Potenzial für Kompression.

Kontext reduziert Unsicherheit

- Entropie sinkt
- Redundanz steigt

TODO:

7. QUELLENCODIERUNG

Anwendungen:

Datenkomprimierung

- Verlustfrei
- Verlustbehaftet

Verschlüsselung

- Symmetrisch
- Asymmetrisch

Kompression ist nur möglich, wenn **Redundanz** vorhanden ist.

Art der Redundanz	Beispiel	Verfahren
Wiederholungen	AAAAAAA	«Run-Length-Encoding (RLE)» (Seite 34)
Ungleiche Wahrscheinlichkeiten	häufige Zeichen	«Huffman-Codierung» (Seite 33)
Muster / Struktur	ABCABCABC	«Lempel-Ziv-Codierung (LZ77)» (Seite 34)

Verfahren zur Datenkomprimierung

Statistische Verfahren	z.B. Huffman-Codierung für die deutsche Sprache – nutzen bekannte Wahrscheinlichkeiten	Eigenart der Daten (Wahrscheinlichkeiten) werden berücksichtigt
Adaptive Verfahren	z.B. Huffman-Codierung mit gemessener Häufigkeitsverteilung – lernen Wahrscheinlichkeiten während der Codierung	

Verfahren zur Datenkomprimierung

Wörterbuchbasierte Verfahren	z.B. LZ77, LZ78, LZW, DEFLATE – nutzen wiederkehrende Muster im Datenstrom	Eigenart der Daten (Wahrscheinlichkeiten) werden nicht explizit berücksichtigt – stattdessen Muster
-------------------------------------	---	--

Ziel der Quellencodierung:

$$R_c \rightarrow 0 \Rightarrow L \approx H(X)$$

7.1. HUFFMAN-CODIERUNG

- Verfahren zur Entwicklung eines präfixfreien Codes mit minimaler mittlerer Codewortlänge L
- Rekursives Verfahren, d.h. der Binärbaum wird nicht von der Wurzel, sondern von den Blättern aus entwickelt

Kerngedanke

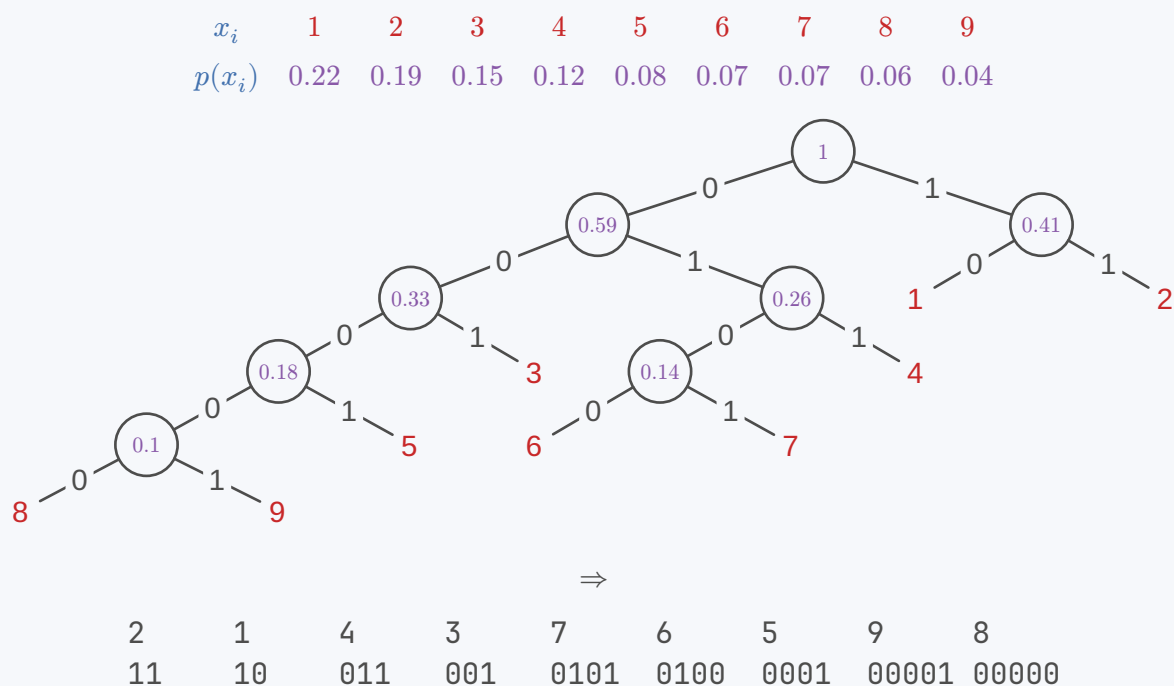
- Häufige Zeichen → kurze Codes
- Seltene Zeichen → lange Codes

Verfahren

- 1) Sortiere Zeichen nach Wahrscheinlichkeit
- 2) Kombiniere die zwei kleinsten
- 3) Ersetze sie durch neuen Knoten
- 4) Wiederhole, bis ein Baum entsteht
- 5) Weise 0 / 1 entlang der Kanten zu

Der Huffman-Code ist nicht eindeutig – aber immer optimal (bzgl. mittlerer Länge).

Beispiel 35



Entropie

TODO:

redundanz etc

$$H(X) \approx 2.89 \text{ Bit}$$

TODO:

bits

7.2. RUN-LENGTH-ENCODING (RLE)

- Wiederholungen werden nicht mehrfach gespeichert, sondern gezählt.
- wird bei vielen Bildformaten benutzt zum Beispiel BMP, PCX und TIFF.
- gehört zu musterbasierten Verfahren
- Spezialfall von Lempel-Ziv
- sehr einfach
- sehr schnell
- keine Wahrscheinlichkeiten nötig

Kerngedanke

- Wiederholungen ersetzen durch: Symbol, Anzahl

TODO:

bits

Gut geeignet für

- lange Wiederholungen
- Bilder (z.B. einfarbige Flächen)
- einfache Muster

Schlecht geeignet für

- zufällige Daten
- häufige Wechsel

Beispiel 36

Quelltext	Codiert
$w = \text{Agggbbbehffgggg}$	$w_c = \text{A3g2beh3f4g}$
$ w = 15$	$ w_c = 11$

7.3. LEMPEL-ZIV-CODIERUNG (LZ77)

TODO:

bits

Kerngedanke

- Muster werden wiedererkannt und ersetzt durch Verweis auf vorherige Sequenz

Grundidee

Zwei Bereiche

- Search Buffer → Vergangenheit
- Look-Ahead Buffer → Zukunft

Grosser Buffer	Kleiner Buffer
Bessere Kompression, weil längere Muster erkennbar	Schneller, weil weniger Speicher
Langsamer	Schlechtere Kompression

Codierung

- Statt Zeichen: Distanz, Länge, Symbol
- Distanz → wie weit zurück
- Länge → wie lang das Muster
- Symbol → nächstes Zeichen

Beispiel 37

Search Buffer							CP	Lookahead Buffer					Out
							a	b	r	a	c	ada...	(0,0,a)
						a	b	r	a	c	a	dab...	(0,0,b)
					a	b	r	a	c	a	d	abr...	(0,0,r)
				a	b	r	a	c	a	d	a	bra	(3,1,c)
		a	b	r	a	c	a	d	a	b	r	a	(2,1,d)
a	b	r	a	c	a	d	a	b	r	a			(7,4,eof)
c	a	d	a	b	r	a							
7	6	5	4	3	2	1							

Tokens

- Distanz = 3Bit
- Länge = 3Bit
- Symbol = 8Bit (ASCII)

Effizienzberechnung

- 14 Bit / Token: $6 \cdot 14 = 84$ Bit
- ASCII: $11 \cdot 8 = 88$ Bit

000 000 01100001	(0,0,a)
000 000 01100010	(0,0,b)
000 000 01110010	(0,0,r)
011 001 01100011	(3,1,c)
010 001 01100100	(2,1,d)
111 100 00000000	(7,4,eof)

7.4. LEMPEL-ZIV-WELCH-KOMPRIMIERUNG (LZW)

Die Lempel-Ziv-Welch-Komprimierung ist ein Verfahren zur verlustfreien Datenkompression, das auf der Lempel-Ziv-Komprimierung aufbaut. Sie verwendet ein Wörterbuch, um wiederkehrende Sequenzen zu identifizieren und durch kürzere Codes zu ersetzen.

7.4.1. Funktionsweise

- 1) Beginne mit einem Wörterbuch, das bereits alle möglichen Einzelzeichen enthält.
- 2) Suche im Wörterbuch die längste Sequenz, die mit den nächsten Zeichen ($n + 1$) der Eingabe übereinstimmt.
- 3) Speichere den Index des gefundenen Wörterbucheintrags.
- 4) Erstelle einen neuen Eintrag im Wörterbuch mit der gefundenen Sequenz, gefolgt vom nächsten Zeichen der Eingabe.
- 5) Verschiebe das Eingabefenster um die Anzahl der codierten Zeichen.
- 6) Wiederhole den Prozess, bis alle Zeichen codiert sind.

Beispiel 38

Gegeben

Zu kodierende Zeichenabfolge: 123123123123

Wörterbuch:

<i>Index</i>	<i>Eintrag</i>
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9

Vorgehen

<i>Buffer</i>	<i>Erkannte Zeichen (Index)</i>	<i>Index: Neuer Eintrag</i>	Neues Wörterbuch:	
123123123123	1 (1)	10: 12	<i>Index</i>	<i>Eintrag</i>
123123123123	2 (2)	11: 23	0-9	Wie bisher
123123123123	3 (3)	12: 31	10	12
123123123123	12 (10)	13: 123	11	23
123123123123	31 (12)	14: 312	12	31
123123123123	23 (11)	15: 231	13	123
123123123123	123 (13)	-	14	312
			15	231

Daraus folgt die codierte Nachricht: 1 2 3 10 12 11 13

7.5. KOMPRESSIONSRATE

$$R(\%) = \frac{L_{\text{komprimiert}}}{L_{\text{original}}} \cdot 100$$

L_{original} = Länge der ursprünglichen Sequenz

$L_{\text{komprimiert}}$ = Länge der komprimierten Sequenz

8. QUBIT

- Superposition
 - Ein Qubit kann gleichzeitig 0 und 1 repräsentieren. Mehrere Qubits können dadurch alle 2^n möglichen Zustände gleichzeitig darstellen.
 - $|\Psi\rangle = \alpha |0\rangle + \beta |1\rangle$
 - $P(0) = |\alpha|^2, P(1) = |\beta|^2$
 - Beispiel
 - $|\Psi\rangle = \sqrt{0.75} |0\rangle + \sqrt{0.25} |1\rangle$
 - $P(0) = 0.75, P(1) = 0.25$
 - Interferenz
 - α, β sind Amplituden. Zwei Qubits können sich gegenseitig beeinflussen, indem die Amplituden mittels Interferenz verstärkt oder ausgelöscht werden
 - Quantenalgorithmen verändern gezielt die Amplituden:
 - richtige Lösungen werden verstärkt
 - falsche Lösungen werden abgeschwächt
 - Verschränkung
 - Verschränkung bedeutet, dass zwei Qubits einen gemeinsamen Zustand besitzen, der sich nicht in zwei unabhängige Einzelzustände zerlegen lässt.
 - Misst man eines der Qubits, ist der Zustand des anderen sofort festgelegt
-

9. CPU

Gesamtübersicht des CPU Zyklus

- 1) Fetch: Instruktion aus RAM lesen
- 2) Decode: Opcode interpretieren
- 3) Operand-Fetch: Speicherwerte laden
- 4) Execute: ALU rechnet
- 5) Writeback: Ergebnis ins Register
- 6) IP erhöhen